git命令思维导图

- 代码仓库
  - 创建仓库
    - 1、进入需要创建代码库的文件夹 — cd 文件路径
    - 2、创建/初始化仓库 — git init
    - 3、拉取远程仓库到本地 — git clone
    - 建议使用git clone
  - 添加文件到仓库
    - 1、添加文件到暂存区
      - 添加单个文件 — git add
      - 添加所有文件 — git add .
      - 会忽略的文件
        - .gitignore中指定的文件会被忽略
        - 空目录
    - 2、提交到本地仓库
      - git commit
        - 填写commit message
        - 保存
      - 不建议使用git commit -m "commit message"
      - 建议提交遵循commit message规范
    - 3、查看工作区状态 — git status
    - 4、对比工作区文件变化
      - git diff
      - 建议将beyond compare配置为diff工具，用于diff以及merge冲突
  - 仓库配置
    - 1、配置全局用户名和邮箱
      - git config --global user.name "[name]" — 比如：git config --global user.name "yousali"
      - git config --global user.email "[email address]"
      - 若是个人开发机可以这样配置，若是公共编译机则不能这样配置
    - 2、配置当前仓库用户名和邮箱
      - git config user.name "[name]"
      - git config user.email "[email address]"
- 代码版本/提交切换
  - 查看过去版本/提交
    - 1、提交详情 — git log
    - 2、提交简介 — git log --pretty=oneline
  - 回退版本/提交
    - 1、回退到当前最新提交 — git reset --hard HEAD
    - 2、回退到上次提交 — git reset --hard HEAD^
    - 3、回退到上n次提交 — git reset --hard HEAD~n
    - 4、回退到某次提交 — git reset --hard commitid
  - 重返未来版本
    - 1、查看历史提交以及被回退的提交
      - git reflog
      - 注意：该记录有时限，且只在本地
    - 2、回到未来版本 — git reset --hard commitid
  - 撤销修改
    - 1、工作区文件撤销
      没有提交到暂存区/没有git add — 撤销修改 — git checkout 文件名
    - 2、暂存区文件撤销
      - 将暂存区文件撤销到工作区 — git reset HEAD 文件
        不带--hard
      - 撤销修改 — git checkout 文件名
    - 3、提交到了版本库 — 参见回退版本/提交
  - 删除文件
    - 1、删除文件
      从版本库中删除文件
      - git rm 文件名
      - 修改后需要提交
    - 2、恢复删除 — 参考撤销修改
    - 3、从版本库中删除文件，但是本地不删除该文件 — git rm --cached 文件名
  - 重命名文件
    - 1、将文件重命名 — git mv
    - 2、将文件夹重命名 — git mv
  - 暂存修改 — 参照分支-暂存修改
  - 忽略文件 — 通过git仓库下的.gitignore文件屏蔽某些中间文件/生成文件
  - 注意：这里的版本均为本地仓库版本
- 分支
  - 创建与合并分支
    - 1、创建分支
      - 仅创建 — git branch 分支名
      - 创建并切换 — git checkout -b 分支名
      - 注意：在本地仓库操作，创建的都是本地分支
    - 2、切换分支 — git checkout 分支名
    - 3、合并分支
      - git merge
        合并某分支到当前分支
      - 注意：合并分支时禁用fast forward — git merge --no-ff 分支名
      - git rebase
        若无特殊需要不建议使用
    - 4、删除分支
      - 删除本地分支
        - 删除未合并分支 — git branch -D 分支名
        - 删除已合并分支 — git branch -d 分支名
      - 删除远程分支
        - 删除远程分支
          - git push origin -d 分支名
          - git push <远程仓库名> -d 分支名
        - 建议界面操作
    - 5、查看分支
      - 查看当前分支 — git branch
      - 查看所有分支信息 — git branch -a
        - 本地分支为本地分支名
        - 远程分支为<远程仓库名>/分支名
    - 6、合并分支，解决分支冲突
      - 将要合并的分支更新到最新
      - 切换到主分支
      - 合并分支
      - 解决合并时的conflict
      - 提交到版本库
      - 合并成功
      - 查看分支状态
        - git log --graph
        - git log --graph --pretty=oneline --abbrey-commit
    - 7、开发完需要提交PR/MR — 通过PR/MR来合并开发分支与主分支
  - 暂存修改
    - 1、暂存工作现场 — git stash
    - 2、恢复工作现场
      - 恢复 — git stash apply
      - 删除 — git stash drop
      - 恢复+删除 — git stash pop
  - 多人协作
    - 1、查看远程库信息
      - 详细 — git remote -v
      - 不详细 — git remote
    - 2、更新/推送远程库
      - 更新远程库信息 — git fetch
      - 将远程库最新修改更新到本地
        - git pull
        - git pull可以认为是
          git fetch+git merge
      - 将本地修改推送到远程库
        - git push
        - git push origin 分支名
    - 3、本地分支与远程分支交互
      - 使用远程分支A创建本地分支
        - git checkout -b A origin/A
        - origin是远程仓库名，若名字一样origin/A可以省略
      - 将本地分支与远程分支作关联
        - git branch --set-upstream A origin/A
        - 提示no tracking information错误
  - 建议开发遵循或者参照git标准工作流，比如git flow、github flow或者gitlab flow
- 代码版本tag
  - 1、查看tag
    - 本地tag — git tag
    - 远程tag — git tag -r
  - 2、操作tag
    - 添加tag
      - 给当前版本添加tag — git tag 标签名
      - 给历史版本添加tag — git tag 标签名 commitid
    - 删除tag
      - 删除本地标签 — git tag -d 标签名
      - 删除远程标签 — git push origin -d 标签名
    - 推送到远端仓库
      - git push origin 标签名
      - 推送所有未提交的tag — git push origin --tags
    - 更新到本地 — git pull origin --tags
  - tag与branch的操作基本一致，因为tag就是一个仅可读的branch
- 其他生僻命令
  - git blame
  - git bisect — 过二分查找定位引入 bug 的变更
  - git relog
  - ...
  - 可以使用git help查看git常用的命令，使用git help -a查看git可用的所有命令